IoT BASED RAILWAY TRACK CRACK DETECTION

ENGINEERING CLINICS (FALL SEM 2024-25)

Submitted to the VIT-AP University

Submitted by:

Members Involved:

Ruthika Narala	23BCE9079
Lasya Priya	23BCE9070
Siri Chandana	23BCE9048
Aare Sarvani	23BCE9075
K Sunayana	23BEC7138
Niksha K.M Teffni	23BEC7122

Under the Guidance of:

Prof. S. SUDHAKAR ILANGO

VIT AP UNIVERSITY

Amaravati,52

2237,Andhra

Pradesh

INDEX:

S NO.	CONTENT	PAGE NO.
01.	PROJECT SUMMARY	03
02.	BLOCK DIAGRAM	04
03.	ORIGINAL VIEW	04-05
04.	PROBLEM STATEMENT	06-07
05.	OBJECTIVES	08-09
06.	METHODOLOGY OF PROJECT	10-11
07.	CONCLUSION	12
08.	FUTURE SCOPE	13-14
09.	REFERENCES & CODE USED IN THE SYSTEM	15-21

PROJECT SUMMARY

Title: IoT Based Railway Track Crack Detection

Project Summary:

Our project, titled **"IoT-Based Railway Track Crack Detection,"** is designed to improve railway safety by detecting cracks or defects on railway tracks in real-time. The system uses IoT technologies to monitor track conditions, aiming to prevent accidents and ensure timely maintenance. The solution is portable, cost-effective, and capable of covering long stretches of railway tracks autonomously.

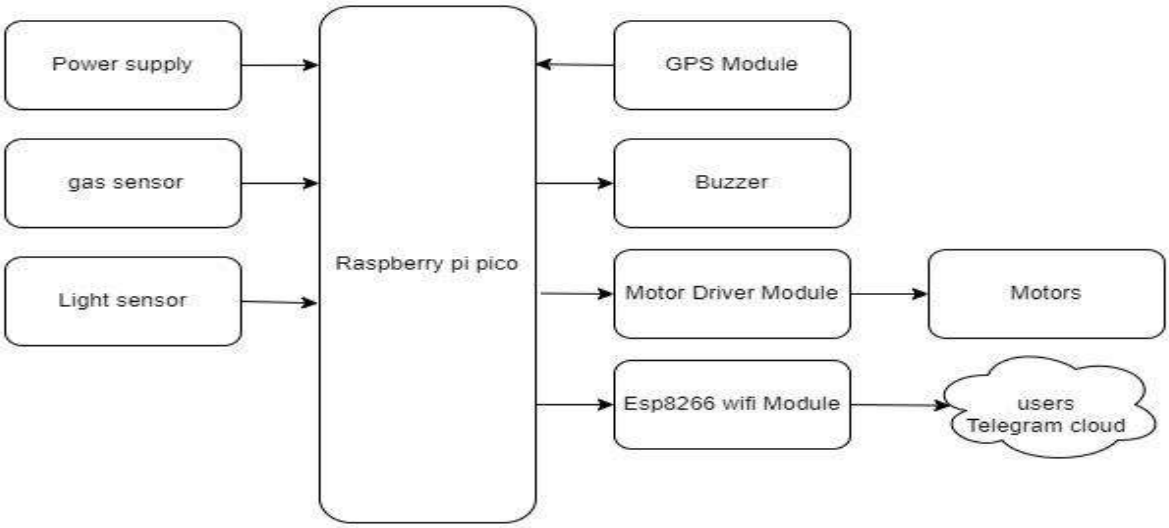
Key Components and Features:

- **Raspberry Pi Pico**: Acts as the central controller, processing sensor data and managing other components.
- **HW-201 IR Sensors**: Detect cracks on the railway track by reflecting infrared light and identifying irregularities in the surface.
- **L298N Motor Driver**: Controls the movement of the DC motors, allowing the system to traverse the tracks.
- **DC Motors**: Provides mobility for the device to move along the railway tracks.
- **GPS Module**: Captures the precise location of any detected crack for accurate reporting.
- **Wi-Fi Module**: Enables real-time data transmission to a remote server for monitoring and alerting authorities.
- **Rashri Multicolour Stabilizer Relay Card**: Ensures stable DC power supply, protecting the system from voltage fluctuations and enhancing reliability.

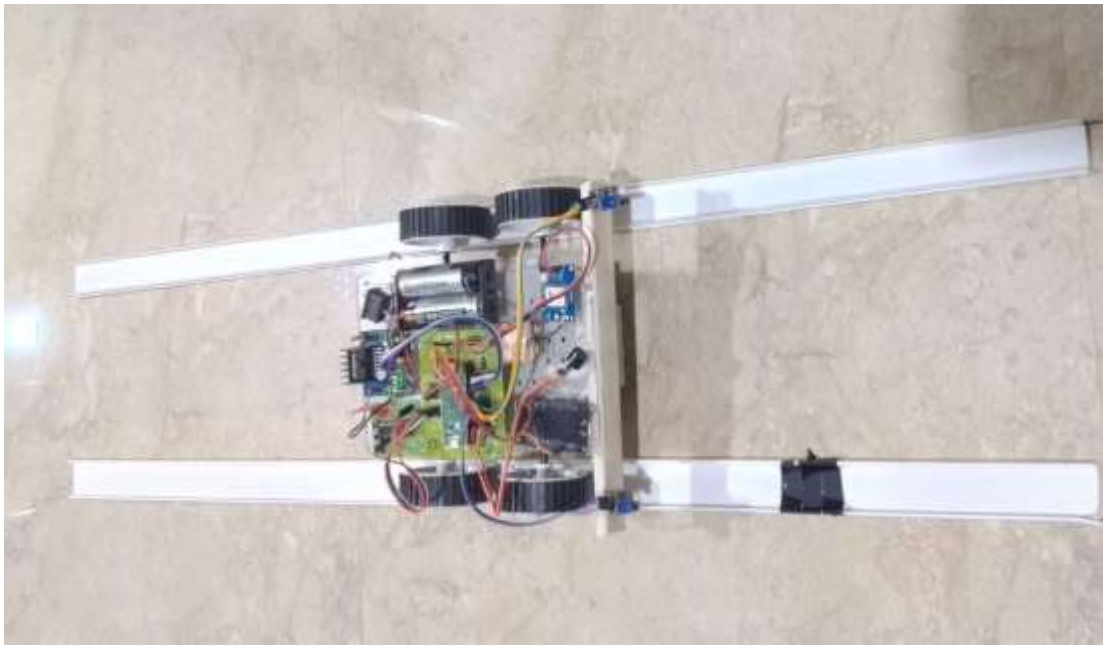
System Workflow:

1. The system moves autonomously along the railway tracks using **DC motors** controlled by the **L298N motor driver**, while the **HW-201 IR sensors** continuously scan for cracks or defects on the track surface.
2. Upon detecting a crack, the **Raspberry Pi Pico** processes the sensor data and captures the precise location using the **GPS module**. This information is logged for accurate fault detection.
3. The system then uses the **Wi-Fi module** to transmit the collected data, including the crack's location, to a remote server for real-time monitoring. The **Rashri Multicolour Stabilizer Relay Card** ensures stable power supply, protecting the device from voltage fluctuations during operation.

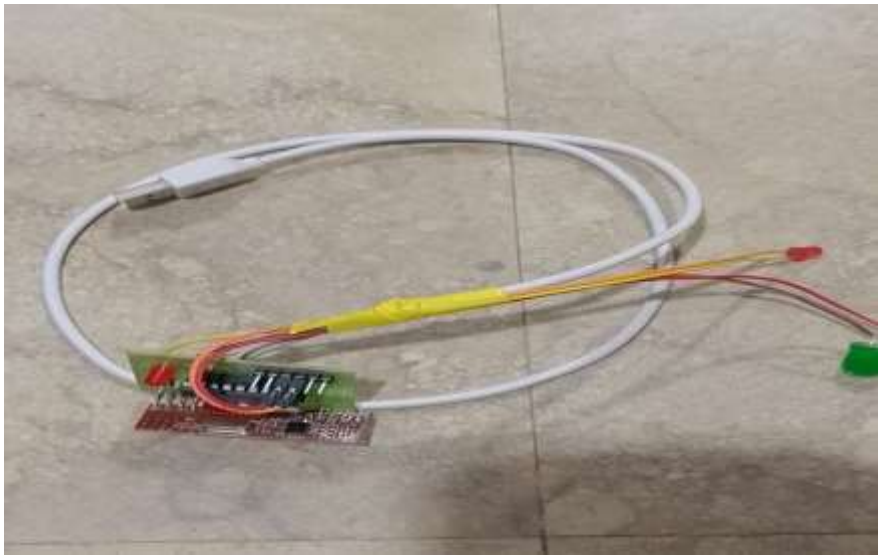
BLOCK DIAGRAM:



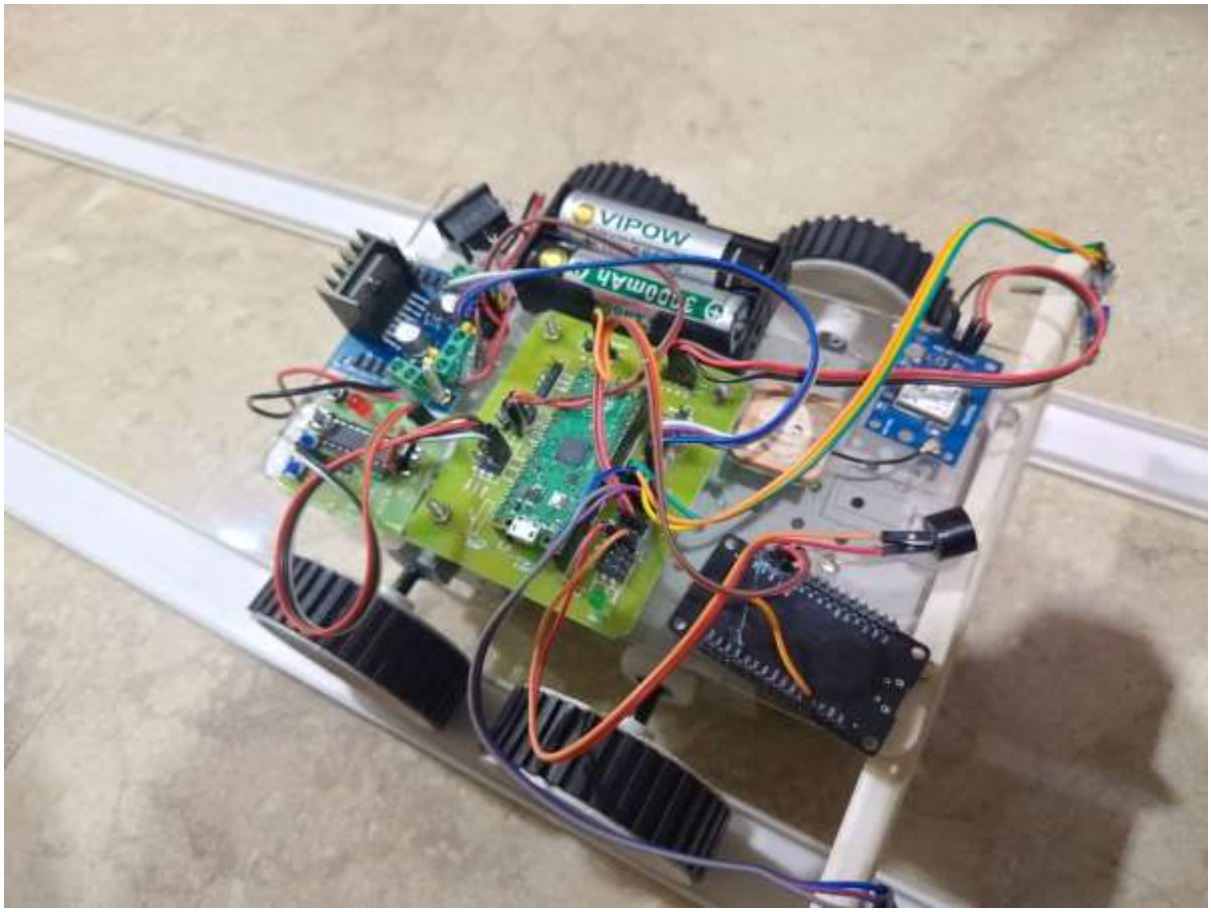
Original view:



Rashri Multicolour Stabilizer Relay Card:



Working Model of Project:



Problem Statement:

Railway track defects, particularly cracks, pose significant safety risks and can lead to derailments, resulting in loss of life and property. The manual inspection of tracks is time-consuming, labor-intensive, and prone to human error, making it inefficient for timely maintenance. There is a pressing need for an automated solution that can efficiently detect track defects in real-time to ensure the safety and reliability of railway operations.

Background:

Railways are one of the most widely used modes of transportation worldwide, especially for long distances and freight. However, the integrity of railway tracks is crucial to prevent accidents. Traditional methods of track inspection rely on periodic manual checks, which may not identify issues promptly. The advent of IoT technology offers an opportunity to automate the detection of track defects, ensuring continuous monitoring and faster response times.

Problem Description:

This project aims to develop an **IoT-based system for railway track crack detection**. The system is designed to traverse railway tracks autonomously, using **IR sensors** to detect cracks and irregularities. Upon identifying a defect, the system uses a **GPS module** to pinpoint the location and a **Wi-Fi module** to send real-time alerts to a central monitoring system. The integration of a **Rashri Multicolour Stabilizer Relay Card** ensures stable power supply, enhancing the system's reliability. This automated solution aims to reduce human error, minimize inspection time, and improve railway safety.

Requirements:

1. Hardware Requirements:

- Raspberry Pi Pico

- HW-201 IR Sensors
- L298N Motor Driver
- DC Motors
- GPS Module
- Wi-Fi Module
- Rashri Multicolour Stabilizer Relay Card
- Power Supply (Battery)

2. **Software Requirements**:

- MicroPython for Raspberry Pi Pico programming
- GPS and Wi-Fi libraries for data handling
- Data processing algorithms for crack detection

3. Functional Requirements:

- Autonomous movement along railway tracks
- Real-time crack detection and alert system
- GPS-based location tracking for detected cracks
- Stable operation under varying voltage conditions

The success of the project will be evaluated based on the following criteria:

1) **Accuracy of Crack Detection**: The system should correctly identify track defects with minimal false positives or negatives.
2) **Real-Time Data Transmission**: The Wi-Fi module should efficiently transmit crack location and status data to a remote server with minimal latency.
3) **Autonomous Operation**: The system must be capable of moving along the tracks autonomously, handling obstacles, and performing consistently in different environmental conditions.
4) **Power Stability**: The integration of the Rashri Multicolour Stabilizer Relay Card should ensure uninterrupted operation, protecting components from voltage fluctuations.
5) **Cost-Effectiveness**: The overall solution should be affordable, making it feasible for widespread implementation across railway networks.
6) **Ease of Deployment**: The system should be easy to set up, maintain, and integrate with existing railway infrastructure.

Objectives:

The project focuses on developing energy-harvesting tiles that convert footstep pressure into electrical energy. The aim is to optimize energy efficiency, ensure long-term durability, and use sustainable materials. The final goal is to test the tiles in real-world conditions and explore their scalability for wider applications.

Develop and Test Energy-Harvesting Tiles:

- Design and prototype tiles that capture mechanical energy from footsteps and convert it into usable electricity. Conduct rigorous testing to ensure that the tiles function reliably under various environmental conditions and foot traffic.

Evaluate Energy Output and Efficiency:

- Measure the energy generated by the tiles and assess their efficiency. Optimize the design to ensure maximum energy output with minimal energy input, making the system suitable for large-scale applications.

Ensure Durability and Safety:

- Test the tiles for resistance to wear, tear, and weather conditions, ensuring they last over time in high-traffic areas. Implement safety features like non-slip surfaces and safe materials to ensure public safety.

Optimize Cost-Effectiveness:

- Analyze production costs to keep the technology affordable for mass production. Explore ways to reduce costs without compromising the efficiency, durability, or safety of the tiles, making them viable for large-scale deployment.

Incorporate Sustainable Materials:

- Use eco-friendly and recyclable materials in the design and production of the tiles. Focus on sustainability by selecting materials that minimize environmental impact while enhancing the energy-harvesting process.

Assess Real-World Application:

- Deploy energy-harvesting tiles in real-world environments to assess their practical performance. Monitor their ability to withstand everyday use, and gather data on their functionality in different public settings.

Explore Scalability and Integration:

- Evaluate the potential to scale up the technology for integration into larger infrastructure, like public transportation systems or urban streets. Investigate how the energy harvested can be integrated into existing grids or used in off-grid applications.

Innovate and Identify Improvements:

- Continuously refine the design and functionality of the tiles. Explore innovative technologies and materials that could further enhance energy efficiency, durability, and cost-effectiveness for future versions of the product.

Methodology of project:

1. Research and Planning

- Conduct thorough research to understand the project scope and objectives.
- Plan the system's architecture, power requirements, and component compatibility.
- Define the project timeline and milestones for smooth execution.

2. Component Selection and Procurement

- Select components based on performance, efficiency, and cost (e.g., energy-harvesting tiles, microcontrollers, sensors).
- Source high-quality components from reliable suppliers to meet project requirements.
- Procure components in alignment with the project timeline.

3. Hardware Setup

- Assemble the energy-harvesting tiles, sensors, microcontroller, and other components.
- Set up the hardware framework, ensuring that components are correctly placed and wired for optimal functionality.
- Ensure compatibility of components for smooth integration.

4. Mechanical and Electrical System Integration

- Integrate the mechanical and electrical systems to ensure proper operation.
- Ensure all electrical connections are secure and functional.
- Test individual components before full integration to minimize issues.

5. Arduino Programming

- Program the Arduino or microcontroller to handle data collection, energy conversion, and control logic.
- Implement code for interaction between sensors, tiles, and power management systems.
- Test and debug the code to ensure it meets functional requirements.

6. Power Management and Efficiency Optimization

- Implement power management techniques to maximize energy efficiency.
- Optimize the system to reduce power losses and ensure effective energy harvesting.
- Monitor and adjust power output based on system requirements.

7. Testing and Validation

- Perform testing in real-world conditions to assess system performance.
- Validate the energy output and system reliability under varying conditions.
- Identify and fix any discrepancies or issues found during testing.

8. System Calibration

- Calibrate sensors and other components for accurate readings.
- Adjust settings to ensure optimal performance.
- Perform final calibration to ensure the system meets the desired output.

9. User Interface Design

- Design a simple, intuitive interface for users to monitor system status.
- Display key data, such as energy output and performance metrics.
- Ensure the interface is easy to interact with and provides real-time updates.

10. Documentation and Reporting

- Document all project processes, including component details, design decisions, and testing results.
- Prepare reports summarizing the project's progress, challenges, and results.
- Ensure all information is clear and easy to understand for future reference.

11. Safety Measures

- Implement necessary safety features like non-slip surfaces and weather resistance.
- Ensure that all electrical components are properly insulated to avoid accidents.
- Conduct safety checks throughout the project development process.

12. Testing and Feedback Iteration

- Gather feedback from initial testing and real-world trials.
- Iterate on the design and functionality based on user input and test results.
- Address issues to improve the system's overall performance and durability.

13. Presentation and Demonstration

- Prepare a final presentation demonstrating the working system.
- Showcase the energy-harvesting functionality and user interface.
- Highlight key features and the real-world application of the system.

14. Knowledge Sharing

- Share project details (schematics, code, findings) on open-source platforms.
- Encourage community collaboration for further development.

Conclusion:

Indian Railways, the world's largest rail passenger network, forms the backbone of the country's transportation infrastructure. This system aims to provide a comprehensive solution to maintaining crack-free railway tracks through an IoT-based detection scheme. The proposed broken rail detection system automatically identifies faulty tracks without human intervention. It detects both cracks and obstacles—whether living or non-living—on the railway tracks. This approach offers several advantages over traditional detection techniques. By pinpointing the precise location of faults, the system enables swift response to prevent accidents, ultimately saving lives and enhancing rail safety.

Future Scope:

1) **Wider Network Integration**:
   The system can be expanded to cover larger sections of railway networks, enabling real-time detection and monitoring of cracks and other track-related issues across the entire railway system. This would contribute to proactive maintenance, reducing downtime and improving overall safety.

2) **Integration with Automated Maintenance Systems**:
   The crack detection system can be integrated with automated railway maintenance systems, allowing for automatic scheduling of repairs when cracks or defects are detected. This would optimize maintenance efforts and reduce human intervention.

3) **Advanced Detection Techniques**:
   Future versions of the system can incorporate additional sensor technologies like ultrasonic or LiDAR (Light Detection and Ranging) to improve the accuracy and depth of crack detection. This would help identify not only surface-level cracks but also deeper structural issues in the railway tracks.

4) **Predictive Maintenance with AI**:
   By integrating machine learning algorithms, the system can analyze historical data to predict when cracks might appear, allowing for predictive maintenance. This could further optimize train schedules and reduce maintenance costs by addressing issues before they cause major disruptions.

5) **Wireless Network and Power Efficiency Enhancements**:
   The system can be enhanced with low-power, long-range wireless communication protocols like LoRaWAN or 5G, allowing for more efficient and long-distance data transmission with minimal power consumption. This will be particularly useful in remote or hard-to-reach areas where connectivity may be limited.

6) **Real-time Alerts and Remote Monitoring:**
The system can be upgraded to provide real-time alerts to railway operators via a mobile or web-based application. This would enable operators to immediately take action if cracks are detected, improving response times and minimizing risks.

7) **Integration with Smart Railway Infrastructure:**
The crack detection system can be incorporated into broader smart railway infrastructure, where it interacts with other sensors (e.g., for monitoring train speed, track alignment, and environmental conditions) to create a comprehensive railway monitoring solution.

8) **Global Adoption in High-Risk Areas:**
The technology can be implemented in regions with high rail traffic or in areas prone to natural disasters (e.g., earthquakes, floods), where track integrity is especially crucial. The system can provide continuous monitoring and ensure railway safety in these critical regions.

9) **Cost Reduction and Commercialization:**
Over time, as the technology matures, the cost of sensors and related components may decrease, making the system more affordable for widespread use in both developed and developing countries. The project could evolve into a commercial product for railway operators globally.

References:

- Idea for the project is acquired from
  1) https://www.hindustantimes.com/india-news/kanpur-crackedtracks-may-have caused-train-to-derail-railway-orders-probe/storyBGv77L0pquleihgjQalvxN_amp.html
  2) https://m.timesofindia.com/india/crack-in-tracks-apparently-led-toderailment railways/amp_articleshow/57913712.cms
- Citations and sources
  1) https://robu.in/ir-sensor-working/
  2) https://micropython.org/
  3) https://medium.com/@kekreaditya/how-does-a-gps-module-work- 81b6892f300a

CODE USED IN THE SYSTEM:

```
from machine import Pin, UART
import utime

buz = Pin(3, Pin.OUT)
lm1, lm2, rm1, rm2 = Pin(10, Pin.OUT), Pin(11, Pin.OUT), Pin(12, Pin.OUT), Pin(13, Pin.OUT)
ls, Rs, rf , rf1= Pin(9, Pin.IN), Pin(8, Pin.IN), Pin(26, Pin.OUT), Pin(27, Pin.OUT)
wifi = UART(0, baudrate=9600, tx=Pin(0), rx=Pin(1))

buff = bytearray(255)

TIMEOUT = False
FIX_STATUS = False

latitude = ""
longitude = ""
satellites = ""
GPStime = ""

def getGPS(wifi):
    global FIX_STATUS, TIMEOUT, latitude, longitude, satellites, GPStime
```

```
    timeout = time.time() + 8
    while True:
        gpsModule.readline()
        buff = str(gpsModule.readline())
        parts = buff.split(',')

        if (parts[0] == "b'$GPGGA" and len(parts) == 15):
            if(parts[1] and parts[2] and parts[3] and parts[4] and
parts[5] and parts[6] and parts[7]):
                print(buff)

                latitude = convertToDegree(parts[2])
                if (parts[3] == 'S'):
                    latitude = -latitude
                longitude = convertToDegree(parts[4])
                if (parts[5] == 'W'):
                    longitude = -longitude
                satellites = parts[7]
                GPStime = parts[1][0:2] + ":" + parts[1][2:4] + ":" +
parts[1][4:6]
                FIX_STATUS = True
                break

        if (time.time() > timeout):
            TIMEOUT = True
            break
```

```
    utime.sleep_ms(500)

def convertToDegree(RawDegrees):

  RawAsFloat = float(RawDegrees)
  firstdigits = int(RawAsFloat/100)
  nexttwodigits = RawAsFloat - float(firstdigits*100)

  Converted = float(firstdigits + nexttwodigits/60.0)
  Converted = '{0:.6f}'.format(Converted)
  return str(Converted)
def setup():
  utime.sleep(1)
  ls.init(Pin.IN)
  Rs.init(Pin.IN)
  lm1.init(Pin.OUT)
  lm2.init(Pin.OUT)
  rf.init(Pin.OUT)
  rm1.init(Pin.OUT)
  rm2.init(Pin.OUT)
  buz.init(Pin.OUT)
  lm1.value(0)
  lm2.value(0)
  rm1.value(0)
  rm2.value(0)
```

```
    rf.value(1)
    rf1.value(1)
cnt=0
def loop():
    utime.sleep(0.2)
    global cnt

    if ls.value() == 0 and Rs.value() == 0:
        sts=0
        lm1.value(1)
        lm2.value(0)
        rm1.value(1)
        rm2.value(0)
        rf.value(1)
        rf1.value(0)

    if ls.value() == 1 and Rs.value() == 0:
        sts=1
        lm1.value(0)
        lm2.value(0)
        rm1.value(0)
        rm2.value(0)
        buz.value(1)
        rf.value(0)
        rf1.value(1)
```

```
    utime.sleep(2)
    buz.value(0)

  if ls.value() == 0 and Rs.value() == 1:
    sts=1
    lm1.value(0)
    lm2.value(0)
    rm1.value(0)
    rm2.value(0)
    buz.value(1)
    rf.value(0)
    rf1.value(1)


    utime.sleep(2)
    buz.value(0)

  if ls.value() == 1 and Rs.value() == 1:
    sts=1
    lm1.value(0)
    lm2.value(0)
    rm1.value(0)
```

```
    rm2.value(0)
    buz.value(1)
    rf.value(0)
    rf1.value(1)

    #print("Crack detected AT ,https://www.google.com/maps/search/?api=1&query=" + str(flat) + "," + str(flon) + "^0")
    utime.sleep(2)
    buz.value(0)

  cnt=cnt+1
  print(cnt)
  if(cnt>20):
    cnt=0;
    wifi.write("700155,408HHB11CXV5WZEE,0,0,SRC 24G,src@internet," + str(sts)+","+ str(16.4971) + "," + str(80.4991) + ",^0")
    print("700155,408HHB11CXV5WZEE,0,0,SRC 24G,src@internet," + str(sts)+","+ str(16.4971) + "," + str(80.4991) + ",^0")

setup()
while True:
  loop()
```

THE

END